"Morris" 5-DOF Robotic Arm

Benjamin McEwan

May 28, 2026

Contents

1 Introduction

The Morris robot is a five degrees of freedom robotic arm designed to be easily 3D printed and assembled and to act as a teaching platform for basic robotics control.

Morris uses five MG996R servo motors as they are common amongst hobbyist robotics, relatively cheap, and easy to control. These are routed into a PCA9685 16 Channel servo driver board which is controlled using an ESP32 microcontroller.

The ESP32 was chosen over the Arduino due to its smaller form factor and the fact that it is more commonly used in final robotics designs than the Arduino, which is primarily a prototyping board. Additionally, the Robot Operating System (ROS) was avoided for Morris as this arm is intended to teach fundamentals and does not require the added complexity of the ROS middleware.

Each of Morris' parts have been designed to be 3D printed easily, quickly, and with minimal supports. The Morris robot was been developed and tested using parts printed in PLA.

2 Software

Included with Morris is a set of four Python scripts and one Arduino IDE script to be used with the Morris robot.

The Arduino IDE script is compiled and uploaded to the robot. Three of the Python scripts use the `serial` library to communicate with the robot via USB. Note that for the Python scripts to run, the Arduino IDE must be closed to allow the ESP32's relevant COM port to remain open.

- `morris_servo-interface.ino` is uploaded to the robot's ESP32 and is used to define a format for serial data sent to the robot in order to control each of the five servos. **This script is important as the robot will not function if it has not been uploaded to its ESP32**.
- `morris_ik-hardware.py` is a basic Python script used to demonstrate the robot's inverse kinematics solving algorithm. Users can manually set the target positon for the robot and its end-effector orientation in the code before running the script. Note that inputting a target outside of the robot's workspace will throw an exception.
- `morris_gui-direct.py` is a Python script which builds a simple user interface that can be used to directly control the position of each servo on the robot.
- `morris_gui-ik.py` is a Python script which builds a simpe user interface that can be used to set and change the target position and end-effector orientation for the robot's inverse kinematics algorithm in realtime with a series of sliders.
- `morris_ik-simulation.py` is a Python script which uses Mujoco to run a simulation of the robot's inverse kinematics algorithm. Note that this script must be run from the `scripts/` directory in order to function.

Table 1: Physical Parameters

Parameter	**Symbol**	**Value** (mm)
Link 2 x length	L_{2x}	100
Link 2 y length	L_{2y}	32
Link 3 length	L_3	90
Link 4 length	L_4	62
Joint 2 Height offset	h	86

3 Analysis

The n links of the robot are taken as L_n with joints J_n. As link 2 has an L-shaped structure it is more specifically denoted by the horizontal and vertical offsets between the joints J_2 and J_3, notated as L_{2x} and L_{2y}. Table 1 shows the physical values of the link length constants for the Morris robot.

3.1 Inverse Kinematics

When calculating the inverse kinematics (IK) of the Morris robot we will consider the robot's spatial frame {s} to be placed at the center of the base of the robot and aligned such that the $-$y direction faces forward, the $-$x direction faces right, and the +z direction faces upwards.

The following covers the IK required to position the robot's end-effector at a point $\vec{p} = (p_x, p_y, p_z)$ in 3D space and an orientation ϕ in the 2D space of the robot's wrist. The robot is kinematically decoupled into three independent systems. The base pivot J_1, the main arm J_2 and J_3, and the wrist J_4.

J_1 is considered a 1R mechanism that acts in the x$-$y plane that only needs to point towards the target point $\vec{p}$, as such the IK angle for J_1 is found via

$$\theta_1 = \text{atan2}(p_y, p_x) \tag{1}$$

J_2, J_3, and J_4 are all considered coplanar. In order to apply IK to these joints, the point $\vec{p}$ must be converted into a point $\vec{p'} = (p'_x, p'_y)$ this plane. The x coordinate of this point can be found as magnitude of the x$-$y plane projection of $\vec{p}$; whilst the y coordinate is simply the z coordinate of $\vec{p}$, therefore

$$p'_x = \sqrt{p_x^2 + p_y^2}$$
$$p'_y = p_z$$

J_2 and J_3 form a 2R mechanism that is decoupled from J_4. The IK equations of this 2R mechanism must therefore be calculated for a point $\vec{P} = (P_x, P_y)$ which is offset from $\vec{p'}$ by the length L_4 at the angle ϕ, the coordinates of this point are found as

$$P_x = p'_x - L_4 \cos(\phi)$$
$$P_y = p'_y - L_4 \sin(\phi) - h$$

where h is the vertical offset of J_2 from the origin of {s} (with a physical value as shown in Table 1). Link 2 posssesses an L-shaped structure, as such an "offset angle" θ_0 is established,

and the length of the effective link L_2 is calculated from its components,

$$\theta_0 = \text{atan2}(L_{2y}, L_{2x})$$
$$L_2 = \sqrt{L_{2x}^2 + L_{2y}^2}$$

The distance between J_2 and the target position $\vec{P}$ is denoted d and calculated as

$$d = \sqrt{P_x^2 + P_y^2}$$

Using the above variables, θ_2 and θ_3 can then be seen via the law of cosines to be

$$\theta_2 = \theta_0 + \text{atam2}(P_y, P_x) + \arccos\left(\frac{L_2^2 + d^2 - L_3^2}{2L_2 d}\right)$$
$$\theta_3 = \arccos\left(\frac{L_2^2 + L_3^2 - d^2}{2L_2 L_3}\right) - \theta_0 - \frac{\pi}{2} \tag{2}$$

From which the angle θ_4 is defined as

$$theta_4 = \phi - (\theta_2 + \theta_3) + \frac{\pi}{2} \tag{3}$$

3.1.1 Implementation of Inverse Kinematics

The calculated angles θ_1, θ_2, θ_3, and θ_4 are the angles between each link and as such they cannot be directly inputted into the robot's servo joints. For this task the auxilliary angles α_n are defined as

$$\alpha_1 = \pi + \theta_1$$
$$\alpha_2 = \pi + \theta_2$$
$$\alpha_3 = \theta_3 - \frac{\pi}{2}$$
$$\alpha_4 = \frac{\pi}{2} - \theta_4$$

It is also recommended that a clamping function is introduced into any implementation of these angles to prevent damagining the servo motors.

3.1.2 Workspace Analysis

The arm has a limited range and as such if a target point $\vec{p}$ is outside of this range then Equations 1, 2, and 3 may fail to produce values. At a basic level, for a point to be in the workspace of the robot the expression

$$(L_2 - L_3) \leq \|\vec{P}\| \leq (L2 + L_3) \tag{4}$$

must hold true. Equation 4 however only describes the workspace according to the inverse kinematics equations and does not fully reflect the physical restrictions placed on the workspace by the robot's design and joint limits. For this reason it may sufficient for ensuring that software implementations of the inverse kinematics do not throw errors, however testing should be done to determine the robot's true workspace and implement the revelant safety features.

4 Bill of Materials (BOM)

Note that jumper cables are not included in the BoM. Washers are also recommended to be placed between some screws and plastic parts

PLA colours are simply recommendations and reflect what was used on the final prototype of the robot.

Table 2: Purchased Parts

Description	**Qty**
MG996R Servo	5
ESP32 Devkit Type-C	1
PCA9685 16-Channel 12-Bit PWM Driver	1
DC5525 to Screw Terminal Connector DC Power Jack (Female)	1

Table 3: Manufactured Parts

Part	**Material**	**Qty**
`base`	PLA - White	1
`link1a`	PLA - White	1
`link1b`	PLA - White	1
`link2`	PLA - White	1
`link2-shell`	PLA - Black	1
`link3`	PLA - White	1
`link3-shell`	PLA - Black	1
`link4`	PLA - White	1
`link4-shell`	PLA - Black	1

Table 4: Fasteners

Description	**Qty**
M3x6 Pan Head Screw	5
M3x10 Pan Head Screw	38